

NewAge Delivery App — Database Guide

Moving from `localStorage` to Supabase / PostgreSQL · written for a new engineer · 2026-08-28

Read this before touching `supabase/migrations/`. It explains *what* the schema is and, more importantly, *why* each choice was made. If a decision here looks arbitrary, it usually isn't — the reasoning is in the grey boxes.

1. Background: how the app stores data today

NewAge is currently a **front-end-only React app**. There is no server and no database. Every piece of data — user accounts, restaurant menus, the shopping cart, orders, delivery tracking — is a JSON string in the browser's `localStorage`. When one part of the app changes data, it calls `window.dispatchEvent(new Event("ordersUpdated"))` and other open components re-read `localStorage` to refresh.

The localStorage keys (the current "database")

Key	Shape	Written by
<code>users</code>	array of <code>{ id, fullName, email, phone, password, role, restaurantName, restaurantAddress }</code>	<code>Login.jsx</code>
<code>vendorCurrentUser</code> / <code>riderCurrentUser</code> / <code>customerCurrentUser</code>	the logged-in user object, one key per role	<code>Login.jsx</code> , <code>storage.js</code>
<code>categories</code>	array of strings, e.g. <code>["Meal","Pizza","Drinks"]</code> — one global list	<code>Menu.jsx</code>
<code>menus</code>	array of menu items, each carrying its own <code>restaurantName</code> / <code>restaurantAddress</code> / <code>vendorName</code>	<code>MenuForm.jsx</code>
<code>cart</code>	flat array of line items <code>{ vendorId, itemId, name, price, quantity, restaurantName, addOns, addedAt }</code>	<code>CartContext.jsx</code> , <code>CartDrawer.jsx</code>
<code>orders</code>	array of food orders — one big blob per order (customer + vendor + rider fields + GPS + items[] + two status strings)	<code>CartDrawer.jsx</code> , <code>VendorOrders.jsx</code> , <code>deliverypool.js</code>
<code>courierOrders</code>	array of package deliveries — similar blob, no line items	<code>CustomerHome.jsx</code> , <code>RiderCourierOrders.jsx</code> , <code>deliverypool.js</code>
<code>useMockData</code>	<code>"true"</code> / <code>"false"</code> — a demo toggle	<code>CustomerHome.jsx</code>

Why this needs to move to a real database

- **Data is per-browser.** A customer on their phone and the vendor on their laptop see completely different data. Nothing is actually shared.
- **Passwords are stored in plain text** in `localStorage`.
- **Race conditions.** `deliverypool.js`'s `acceptDelivery()` reads the whole orders array, edits it in JavaScript, and writes it back. If two riders accept the same delivery at nearly the same moment, both can "win" — there's no lock.
- **No integrity.** Nothing stops a menu item pointing at a vendor that doesn't exist, or an order line with a negative quantity.

What Supabase gives us

PostgreSQL

a real relational database. Tables, foreign keys, constraints, transactions.

Auth

manages email + password (hashed), sessions, password reset. Replaces the `users` key and the `*CurrentUser` keys.

Row Level Security (RLS)

rules written *in the database* deciding which rows each user may read or write. Replaces the scattered `if (currentUser.role !== 'vendor')` checks in the components — those can be bypassed by anyone with dev tools; RLS cannot.

Realtime

the client subscribes to table changes and gets pushed updates. Replaces `window.dispatchEvent`, and works across devices, not just tabs.

RPC

PostgreSQL functions the client calls like an API. We use these for multi-step changes that must be atomic (checkout, accepting a delivery).

2. The three migration files

File	Contents	Why it's separate
<code>0001_initial_schema.sql</code>	8 tables, 6 enums, foreign keys, <code>updated_at</code> triggers, the signup→profile trigger	The shape of the data. Rarely changes once stable.
<code>0002_row_level_security.sql</code>	RLS policies for every table + the <code>auth_role()</code> helper	Security rules change more often than table shapes; keeping them apart makes review easier.
<code>0003_rpc_functions.sql</code>	14 functions: checkout, accept/cancel/progress a delivery, GPS push, mark paid	Application logic. Most likely to be iterated on.

Migrations run in filename order and only once each — that's how `supabase db push` knows what's already applied. Never edit a migration that has already run on a shared database; add a new `0004_...` instead.

3. The tables, and the reasoning behind each

3.1 `profiles` was: `users + *CurrentUser`

One row per user. Holds name, phone, role, and (for vendors) restaurant name and address.

Why `profiles.id` equals `auth.users.id`: Supabase Auth owns the `auth.users` table (email, hashed password). We can't add our own columns to it. The standard pattern is a `profiles` table whose primary key is *also* a foreign key to `auth.users(id)`. Same UUID, one-to-one. So "the login" and "the profile" are the same identity, and `ON DELETE CASCADE` means deleting the auth user cleans up the profile automatically.

Why a trigger creates the profile: when someone signs up, Supabase only writes to `auth.users`. The `on_auth_user_created` trigger fires right after and inserts the matching `profiles` row, copying `full_name`, `role`, `phone`, `restaurant_*` out of the sign-up metadata the client passed. Without this, every new user would have a login but no profile.

Why the `vendor_has_restaurant` CHECK constraint: `MenuForm.jsx` refuses to add a menu item if the vendor has no `restaurantName` on file. We move that rule into the database: a row with `role = 'vendor'` must have both `restaurant_name` and `restaurant_address`. Customers and riders leave them null.

3.2 `categories` was: `categories (one global string[])`

Why it's now per-vendor: the old app had a single global list of category names shared by every restaurant. That's wrong — a pizza place and a smoothie bar shouldn't share one "Drinks" category they both edit. The table has `vendor_id` plus `unique (vendor_id, name)`, which also reproduces the old "Category already exists" check as a database constraint.

3.3 `menu_items` was: `menus`

Why the restaurant fields are gone: every old menu object carried `restaurantName`, `restaurantAddress`, `vendorName` — duplicated on every single item — so the customer pages could build restaurant listings from the menu array alone. In a relational database that's a single `vendor_id` foreign key; you `JOIN` to `profiles` to read those fields. One source of truth: rename the restaurant once, every menu item reflects it.

Why `category` is plain text, not a foreign key to `categories`: `MenuForm.jsx` writes a free string, and the old data lets an item keep its category even after the vendor deletes that category. Making it a hard FK would break that. If you'd rather have integrity here, swap to `category_id` uuid references `categories(id)` — it's a one-line change, flagged in the file.

Why price numeric(10,2) not float: money must never be a floating-point number — `0.1 + 0.2 != 0.3` in float. `numeric` is exact. The `check (price >= 0)` stops negative prices.

3.4 `carts` + `cart_items` was: `cart`

Why split into two tables: the old `cart` was a flat array of line items. Normalised, that's a `carts` row (one per customer, enforced by `unique (customer_id)`) and many `cart_items` rows.

Why the single-vendor trigger: at checkout the app only ever looks at `cart[0].vendorId` — it assumes the whole cart is from one restaurant. The `cart_items_single_vendor` trigger enforces that assumption: adding an item from a different vendor raises an error. Better to fail loudly at add-time than silently drop items at checkout.

Why unit_price is stored on the cart item: a price snapshot. If the vendor re-prices a dish while it's sitting in someone's cart, the customer still checks out at the price they saw. `add_ons` stays `jsonb` because add-ons are still a hardcoded generic list (`src/data/addOns.js`), not their own table yet.

3.5 `orders` + `order_items` was: `orders (food)`

The old order object was one large blob mixing four concerns: who ordered, which restaurant, the rider half, and GPS for both parties — plus an `items[]` array and two separate status strings.

Why two status columns: the app genuinely tracks two independent things. `status` (`order_status` enum: Placed / Preparing / Ready / Cancelled) is the *restaurant's* progress, set in `VendorOrders.jsx`. `delivery_status` (`food_delivery_status` enum: Waiting... / Accepted by Rider / Picked Up / Out for Delivery / Delivered) is the *rider's* progress. The customer's Orders page shows whichever one is relevant. Keeping them as two enums means the UI can't be handed a status it doesn't know how to render.

Why the rider columns are nullable: `rider_id`, `rider_name`, `rider_earnings`, `rider_latitude/longitude` are all null until a rider accepts the job. That's the "unassigned" state. CHECK constraints keep it honest — e.g. `rider_paid` can't be true if `rider_id` is null.

Why rider_earnings lives on the order, set by the vendor: in `VendorOrders.jsx` the vendor is prompted for the delivery fee at the moment they mark the order "Ready" — that's what releases it to riders. So the fee is a property of the order, decided by the vendor. (Courier orders are the opposite — see below.)

Why order_items snapshots name and unit_price: an order is a receipt. It must always show what was actually bought and charged, even years later after the menu item is deleted or re-priced. `menu_item_id` stays as a foreign key with `ON DELETE SET NULL` — the live link is nice to have, but the snapshot is the record.

3.6 `courier_orders` was: `courierOrders (packages)`

Package delivery. No line items — one package, one pickup address, one drop-off address.

Why one status column, not two: unlike food orders there's no restaurant in the middle. One `courier_status` enum (Waiting for Rider / Rider Assigned / Out for Delivery / Delivered) covers the whole lifecycle.

Why the rider sets the fee here: in `RiderCourierOrders.jsx` the rider is prompted for their fee when they *accept* the job — they're quoting a price. So `delivery_fee` is null until assignment, then set by the rider. The CHECK constraint `courier_fee_set_when_assigned` enforces exactly that.

4. Foreign keys — and why each `ON DELETE` rule

```
auth.users (Supabase Auth — owns email + hashed password)
  | id
  ▼ 1-1
profiles
  | id (PK, = auth.users.id) ON DELETE CASCADE
  |
  |< categories.vendor_id      ON DELETE CASCADE
  |< menu_items.vendor_id     ON DELETE CASCADE
  |< 1 carts.customer_id      ON DELETE CASCADE
  |< orders.customer_id       ON DELETE SET NULL
  |< orders.vendor_id         ON DELETE RESTRICT
  |< orders.rider_id          ON DELETE SET NULL
  |< courier_orders.customer_id ON DELETE SET NULL
  |< courier_orders.rider_id   ON DELETE SET NULL

carts
  |< cart_items.cart_id       ON DELETE CASCADE
menu_items
  |< cart_items.menu_item_id  ON DELETE CASCADE
  |< order_items.menu_item_id ON DELETE SET NULL
orders
  |< order_items.order_id     ON DELETE CASCADE
```

The three rules, and when to use which

ON DELETE CASCADE — "the child cannot exist without the parent"

Delete a vendor → their categories and menu items are meaningless, so they go too. Delete a cart → its items go. Delete a menu item → remove it from everyone's cart (you can't buy something that's gone).

ON DELETE SET NULL — "this is a historical record that must outlive the person"

An order must survive its customer or rider leaving the platform — the vendor still needs their sales history, accounting still needs the record. The FK becomes null, but `orders.customer_name`, `orders.rider_name`, `order_items.name/unit_price` are stored as plain-text **snapshots** precisely so the row still reads correctly with a null FK.

ON DELETE RESTRICT — "refuse the delete while children exist"

Only on `orders.vendor_id`. You should not be able to delete a restaurant that still has orders attached — close it out first. This is a safety catch against accidental data loss.

Snapshot columns are denormalised on purpose. `orders.customer_name`, `orders.customer_phone`, `orders.rider_name`, `order_items.name`, `order_items.unit_price`, `cart_items.unit_price`, `courier_orders.customer_name / customer_phone / rider_name` all copy a value that also lives behind a foreign key. Normally duplication is a smell. Here it's deliberate: **the FK is the live link; the snapshot is the receipt**. A receipt doesn't change when the menu changes.

5. The rider "offer pool" — how it's reproduced

`src/utils/deliverypool.js` is the heart of the rider experience. It merges food orders and courier orders into one list of "available deliveries", offers them one at a time, and lets a rider hold a batch. Two mechanisms reproduce it:

1. **Partial indexes** that match exactly the "available" predicate:

- `orders_offer_pool_idx` — `WHERE status = 'Ready' AND rider_id IS NULL`
- `courier_orders_offer_pool_idx` — `WHERE status = 'Waiting for Rider' AND rider_id IS NULL`

A partial index only indexes rows matching the condition — small and fast, since the pool is a tiny fraction of all orders.

2. **RLS clauses** that let any user with `role = 'rider'` `SELECT` rows matching those same predicates. So the dashboard can query the pool directly and safely.

Why the accept logic moved into an RPC: the old `acceptDelivery()` did read-modify-write in JavaScript — fetch the array, find the order, set `riderId`, write the array back. Two riders running that at the same time both read "no rider yet" and both write themselves in. The RPCs `accept_food_delivery()` / `accept_courier_delivery()` do `SELECT ... FOR UPDATE` inside a transaction, which locks the row. The second rider's call finds the row already taken and returns `false` — same return contract the old function had, now actually correct.

6. Row Level Security — who can see and do what

With RLS enabled, the front end can talk to the database directly using the public "anon" key, and PostgreSQL itself enforces access. A malicious user with dev tools still can't read another customer's orders, because the rule runs server-side.

Table	Read	Write
<code>profiles</code>	any signed-in user (needed to show names / restaurant info)	only your own row
<code>categories</code> , <code>menu_items</code>	any signed-in user (customers browse menus)	only the owning vendor
<code>carts</code> , <code>cart_items</code>	only the owning customer	only the owning customer
<code>orders</code>	the customer, the vendor, the assigned rider — or any rider while it's an open offer	customer creates; vendor updates restaurant-side; rider claims & progresses
<code>order_items</code>	whoever can see the parent order	the customer, on their own order
<code>courier_orders</code>	the customer, the assigned rider — or any rider while unassigned	customer creates & can cancel while unassigned; rider claims & progresses

Why the `auth_role()` helper function: policies often need "is the current user a vendor?". Reading `profiles.role` inside a policy on another table is fine, but doing it in a policy on `profiles` itself causes infinite recursion. A `SECURITY DEFINER` function bypasses RLS for that one lookup and breaks the cycle.

Why some writes go through RPC instead of a plain `UPDATE` policy: RLS gates whole rows, not columns. We don't want a rider able to edit an order's `total`, only its `delivery_status`. The RPC functions (`SECURITY DEFINER`) touch exactly the right columns and check ownership, so the row-level policy can stay coarse.

7. Replacing `window.dispatchEvent` with Realtime

```
supabase
  .channel('orders')
  .on('postgres_changes',
    { event: '*', schema: 'public', table: 'orders' },
    () => reloadOrders()
  ).subscribe();
```

Every place the code currently calls `window.dispatchEvent(new Event("ordersUpdated"))` becomes a Realtime subscription. Enable it per table (Dashboard → Database → Replication, or `ALTER PUBLICATION supabase_realtime ADD TABLE orders;`) for `orders`, `courier_orders`, `menu_items`, `cart_items`. Unlike the old events, this works across devices.

8. Decisions still open — discuss before building the client

Multi-role accounts. **needs a call** The current app lets one person be a customer, a vendor, and a rider simultaneously (three separate `*CurrentUser` keys). This schema gives each login **one** `role`. Options:

1. Add a `user_roles (user_id, role)` join table — one login, many roles. Costs: every RLS policy changes from `profiles.role = 'x'` to an `EXISTS` subquery.
2. One role per account. Someone who is both a vendor and a rider keeps two accounts. This is how DoorDash and Uber Eats actually work. **Recommended** unless there's a real product reason not to.

Nothing else in the schema depends on the choice — `role` is only read by the RLS policies and `auth_role()`.

`menu_items.category` as text vs. foreign key. **by design, revisit later** Currently free text, to match `MenuForm.jsx` and the old data (an item keeps its category string after the category is deleted). Switch to `category_id` uuid references `categories(id)` if you want the database to guarantee every menu item points at a real category.

9. Not modelled yet — future migrations

- **Per-item add-ons.** `src/data/addOns.js` is a hardcoded generic list. When vendors define their own, add `menu_item_add_ons (id, menu_item_id, name, price)` and change `cart_items.add_ons` / `order_items.add_ons` from `jsonb` to a proper child table.
- **Payments.** No payment step exists in the app today. A `payments` table would reference `orders.id` / `courier_orders.id`.
- **Rider batches.** `MAX_BATCH_SIZE` batching lives entirely in the dashboard's React state. Server-side would be a `delivery_batches` table with `orders.batch_id` / `courier_orders.batch_id`.
- **Ratings, saved addresses, promo codes.** None exist yet.

10. How to apply the migrations

```
npm i -g supabase          # install the CLI
supabase init               # once - creates supabase/config.toml
supabase link --project-ref <ref> # <ref> is in your project's dashboard URL
supabase db push            # runs 0001, 0002, 0003 against the project

# to experiment locally instead (needs Docker):
supabase start              # local Postgres + Studio UI
supabase db reset           # re-applies all migrations from scratch
```

Rule: once a migration has run on a shared database, never edit it. Fix mistakes with a new numbered migration. `supabase db push` tracks which files it has already applied.

Generated from `supabase/README.md` and the migration files. NewAge Delivery App — internal engineering documentation.